



Project Report: Open Source Modular Controller

Fall 2025

Prepared by:

Ethan Dwyer

Devon Flanery

Gabriel Fosdick

Brady Johnston

Andres Salinas

Stephen Venerable



Table of Contents:

Table of Contents:	1
Executive Summary:	2
Core Technology	2
Innovation	2
Development Highlights	2
Impact	2
Main Product Image:	3
Project Description:	3
Methodology:	4
System Architecture	4
USB HID Implementation	4
Component Selection and Design Considerations	5
Mechanical and Ergonomic Design	5
Summary	5
Bill of Materials:	6
Tools used:	8
Software	8
Hardware	8
Assembly:	9
Characterization Data:	16
Input testing	16
Ergonomic design	17
Hardware continuity	17
Sponsors/Acknowledgements:	18
References:	18

Executive Summary:

The goal of this project is to design and develop an open-source modular game controller compatible with modern gaming and computing systems. The controller features interchangeable input modules that enable users to customize both the layout and physical placement of controls based on game genre, personal preference, or accessibility needs. By supporting a variety of input types and configurations, the system can be tailored for diverse users and applications, from video games to robotics control. Once assembled, the controller connects directly to standard devices using USB and is recognized as a Human Interface Device (HID) without additional drivers.

Core Technology

At the foundation of the system is the TinyUSB library, which allows communication within the USB HID framework. This approach eliminates the need for dedicated drivers and ensures compatibility across Windows, macOS, and Linux platforms, providing a plug-and-play experience for end users.

Innovation

The project's defining innovation lies in its fully modular and spatially flexible architecture. Each controller consists of removable button and joystick modules that attach to detachable receiver boards. These receiver boards can be placed anywhere on a custom chassis, giving users the freedom to design their own controller shapes and ergonomics. This allows complete reconfiguration of both the electronic and physical layout, something not possible with commercial controllers. The system is designed to be approachable for those with little to no electronics experience while remaining highly extensible for makers and developers.

Development Highlights

Over the course of the semester, the team developed the first-generation receiver boards and two initial modules: a button input and a joystick. The base system successfully interfaced with a Windows PC and functioned in real-world gaming scenarios. Future work will expand on this foundation by adding module identification, improving the mechanical design, and refining the inter-module communication protocol to support more advanced features.

Impact

This project represents a meaningful advancement in customizable controller design. Its modular and repositionable hardware offers new possibilities for makers, competitive gamers, and individuals with physical disabilities, enabling truly personalized ergonomics and input arrangements. By combining open-source firmware with hardware flexibility, the system empowers users to create, share, and refine their own control interfaces, making high-performance and adaptive hardware accessible to everyone.

Main Product Image:



Figure 1: Main Controller Image

Project Description:

This semester's focus was the design and implementation of a functional prototype for a modular, open-source game controller compatible with modern computing systems. The primary goal was to develop a controller system in which every major input—joysticks, buttons, and future modules—can be physically rearranged or replaced, giving users the freedom to build a controller layout that matches their play style or accessibility needs.

Development began with a breadboard-based proof of concept used to verify signal routing, input functionality, and USB communication. Once validated, the team progressed to designing custom printed circuit boards (PCBs) for the joystick and button modules, the receiver boards, and an ESP32-based carrier board serving as the main controller.

A 3D-printed enclosure was then modeled and fabricated to house the assembled electronics. Each receiver board connects to the module boards through standardized headers, allowing the modules to be placed anywhere within a custom chassis. This spatial flexibility is the defining feature of the system, enabling users to create their own controller shapes and layouts to suit personal or accessibility requirements.

The controller's firmware utilizes the TinyUSB library to manage communication through the USB Human Interface Device (HID) framework, ensuring plug-and-play functionality without custom drivers. The current prototype successfully interfaces with Windows systems and is fully recognized as a standard game controller. Although additional inputs such as bumpers and

triggers are planned for future iterations, the completed prototype establishes a robust hardware and software foundation for a fully modular and reconfigurable controller platform.

Methodology:

System Architecture

The modular controller system was built around the ESP32-S3 microcontroller, chosen for its native support of both USB and Bluetooth HID (Human Interface Device) communication. This capability allows the controller to function as a USB peripheral without requiring custom drivers while also providing wireless compatibility for future iterations.

The system consists of a main controller board, several receiver boards, and interchangeable input modules such as joysticks and buttons. Each receiver board connects to the controller board via standardized pin headers that carry both power and data. Because the receivers can be placed anywhere within a custom chassis, users can define their own controller layout and ergonomics. This modular, spatially flexible design is the foundation of the project's innovation and enables complete customization of control configurations.

USB HID Implementation

Communication between the controller and host system relies on the USB HID standard, which defines how input devices transmit data to computers and consoles. When a device is connected, the host begins a process called enumeration, during which it requests a series of descriptors from the device. These descriptors identify the device type, power requirements, and data structure:

- Device Descriptor: Specifies the vendor ID, product ID, and device class.
- Configuration and Interface Descriptors: Define endpoints, interfaces, and required power.
- HID and Report Descriptors: Describe the logical format of inputs such as buttons and joystick axes.
- String Descriptors (optional): Provide human-readable information, such as product name or manufacturer.

Once the enumeration sequence is complete, the device begins sending HID reports, which contain button states and analog axis values. These reports are continuously exchanged with the host to provide real-time input recognition. Using the TinyUSB library, the ESP32-S3 handles all enumeration, descriptor generation, and report formatting internally, allowing the controller to be recognized as a standard gamepad on Windows without additional drivers.

Component Selection and Design Considerations

Component selection emphasized functionality, modularity, and cost efficiency. The ESP32-S3 was selected for its integrated USB OTG interface, Bluetooth LE support, and sufficient processing resources for handling multiple modular inputs. Standard pin headers were chosen as the initial connection method between modules and receiver boards due to their availability, affordability, and mechanical stability. Future versions may transition to magnetic pogo-pin connectors to simplify module swapping and improve durability.

All printed circuit boards (PCBs) were designed in-house and fabricated through a low-cost prototyping service. Careful attention was given to connector orientation, trace routing, and power distribution to ensure reliability and ease of assembly.

Mechanical and Ergonomic Design

The physical design was driven by a goal of consistent module sizing and ergonomic balance. The joystick module established the baseline diameter, which the button module matched for symmetry and ease of manufacturing. The 3D-printed enclosure was designed to house the main carrier board and four input modules, providing sufficient internal space for wiring while maintaining a compact, comfortable form factor. Extended side grips were added to improve handling during gameplay without compromising the modular placement of receivers.

Summary

This methodology established a repeatable framework for modular hardware development: from microcontroller selection and USB communication setup to enclosure design and fabrication. The resulting prototype demonstrates reliable USB HID communication, functional modular inputs, and a flexible architecture ready for continued expansion and refinement.

Bill of Materials:

Currently, there are a few different controllers that can be made. The main difference is the amount and difficulty of soldering you have to do. The first is a custom ESP32 carrier board for the ESP32-S3-WROOM-1U module with a custom mounting method. The second is the one using the ESP32-S3-DevKitC-1, which was used for the shell design. This reduces the complex soldering down to only headers, buttons, capacitors, and resistors.

Table 1: Bill of materials

Part name	Desc	Qty.	Total	Link
MCU Carrier Board				
ESP32-S3-WROOM-1U-N4	ESP32-S3 module	1	\$5.06	Digikey.com
1N5819WS	Reverse-current protection diode	1	\$0.10	Digikey.com
AZ1117CH-3.3TRG1	LDO regulator	1	\$0.16	Digikey.com
PRTR5V0U2X,215	ESD protection	1	\$0.47	Digikey.com
USB-CB16HTBTR	USB connector	1	\$1.05	Digikey.com
603_1208 metric	10 uF capacitor	3	\$0.30	Digikey.com
603_1208 metric	.1 uF capacitor	4	\$0.32	Digikey.com
SML-D12U1WT86	Red LED	1	\$0.12	Digikey.com
SKQGABE010	Button	1	\$0.31	Digikey.com
603_1208 metric	5.1 kΩ resistors	2	\$0.32	Digikey.com
603_1208 metric	1 kΩ resistors	1	\$0.10	Digikey.com
603_1208 metric	10 kΩ resistors	1	\$0.10	Digikey.com
DF40C-20DP-0.4V(51)	Plug	2	\$1.38	Digikey.com
	Subtotal:	20	\$9.79	
MCU interposer				
DF40C-20DS-0.4V(51)	Receptical	2	\$1.16	Digikey.com
603_1208 metric	10 uF capacitors	2	\$0.20	Digikey.com
603_1208 metric	.1 uF capacitors	2	\$0.16	Digikey.com
SKQGABE010	Button	3	\$0.93	Digikey.com
LTST-C193TBKT-5A	Blue LED	4	\$0.72	Digikey.com
603_1208 metric	1 kΩ resistors	4	\$0.40	Digikey.com
	Subtotal:	17	\$3.57	
Dev board interposer				
ESP32-S3-DevKitC-1-N8R 8 Development Board	ESP32-S3 dev board	1	\$15.00	Amazon.com

Part name	Desc	Qty.	Total	Link
603_1208 metric	10 uF capacitors	2	\$0.20	Digikey.com
603_1208 metric	.1 uF capacitors	2	\$0.16	Digikey.com
SKQGABE010	Button	3	\$0.93	Digikey.com
LTST-C193TBKT-5A	Blue LED	4	\$0.72	Digikey.com
603_1208 metric	1 kΩ resistors	4	\$0.40	Digikey.com
PPTC221LFBN-RC	01x22 female pin headers	2	\$2.40	Digikey.com
	Subtotal:	18	\$19.81	
Input Boards				
CC0603KRX5R8BB105	1 uF capacitors	4	\$0.40	Digikey.com
9032	Joysticks	2	\$10.50	Digikey.com
B3F-1000	Button	12	\$2.88	Digikey.com
PRPC040SACN-RC	header pins 1x40 pin set	2	\$1.68	Digikey.com
	Subtotal:	20	\$15.46	
Receiver Boards				
PPPC041LFBN-RC	1x4 pin header sockets	16	\$6.08	Digikey.com
PRPC040SBAN-M71RC	header pin right angle 1x40 pins	0	\$0.00	Digikey.com
26 AWG Wire, 26 AWG Silicone Electrical Wire -SCHDRA 26 Gauge Tinned Copper Wires	26 AWG stranded wire	1	\$9.99	Amazon.com
	Subtotal:	17	\$16.07	
PCBs				
Custom MC Board	JLC makes 5 # in qty. is # used	1	\$5.00	Link to Gerber files
MC carrier board	JLC makes 5 # in qty. is # used	1	\$5.00	Link to Gerber files
MC dev board carrier	JLC makes 5 # in qty. is # used	1	\$5.00	Link to Gerber files
Button Board	JLC makes 5 # in qty. is # used	2	\$5.00	Link to Gerber files
Joystick Board	JLC makes 5 # in qty. is # used	2	\$5.00	Link to Gerber files
Receiver Board	JLC makes 5 # in qty. is # used	4	\$5.00	Link to Gerber files
	Sutotal:	11	\$30.00	

Part name	Desc	Qty.	Total	Link
Shell				
3D Printer Filament	PLA	1	\$2.50	link to step file
Heat-set inserts	150 PCS M3 Brass Inserts	12	\$5.69	amazon.com
M3 skewers	Taiss 620 PCS M3 Screw Kit	12	\$6.99	amazon.com
	Subtotal:	25	\$15.18	
Total		103	\$109.88	

Tools Used:

Software

Table 2: Software used

Name	Description	Price	Link
GitHub Desktop	Used to share updates and versions.	Free	GitHub.com
Arduino IDE	Used as the programming platform.	Free	Arduino.cc
KiCAD	The program was used for creating the PCB.	Free	KiCAD.com
OnShape	Used for the frame design.	Free	OnShape.com
Ultimaker Cura	Used as a slicer for the 3-D printed glasses frame.	Free	Ultimaker.com
Total Price			

Hardware

Table 3: Hardware used

Name	Description	Price	Link
3D-printer	Ender-3 printer with PLA 1.75mm used for the frames of the glasses.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	3D-Printer

Name	Description	Price	Link
Solder Iron	Used for touch-ups and adding components.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Solder Gun
Heat gun	The heat gun was used to help get the ESP32 on the boards and align some of the components.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Heat Gun
Multimeter	The Multimeter was from the lab and was used to test the PCB.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Multimeter
Sandpaper	The sandpaper was used to clean up the edges of the PCB. A finer grit would be Ideal for this situation.	These parts come in a variety of prices, qualities, and quantities, or they may be found at a local makerspace	Sandpaper
Tweezers	The tweezers were used to hold components in place when soldering.		
Total Price		Varies	

Assembly:

1. Gather Materials
 - a. Refer to the BOM
 - b. Download the main GitHub repo branch ([link](#))
2. Order PCBs
 - a. Use our PCB manufacturer; the Gerber files are provided in the GitHub repository.
 - b. You can get the board preassembled in this iteration. There are two different board sets available for order.
This version uses the version that uses the DevKitC module, not the custom ESP32 board.
3. 3D Print the shell and module pieces.
4. Prep the PCB
 - a. Sand edges
 - b. Clean with isopropyl alcohol
5. Solder components (if necessary)

In this production, blank boards were ordered, and most components were hand-soldered with only a soldering iron.

 - a. Custom ESP board

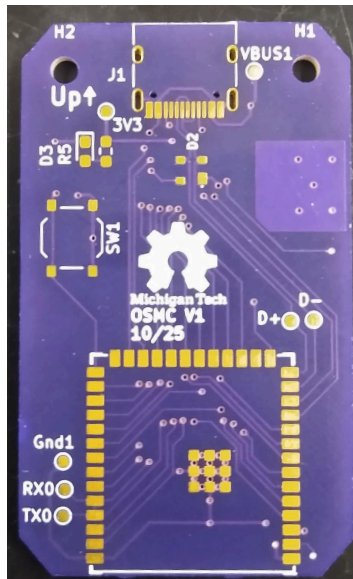


Figure 2: Custom ESP32-S3 Board

- i. Solder the headers to the bottom of the board
Take your time; use flux and solder wick.
- ii. Solder on the rest of the components on the bottom of the board, working up from the smallest/lowest profile component up. This will give you room to work on the various components.
- iii. Solder on the USB port with the same method as the headers

- iv. Solder the ESP32-S3 module
 - 1. Pre-tin the module pads and PCB pads by placing a little solder on the pads.
 - 2. Heat using hot air above and below until the module self-aligns and settles. May need to hold the module down with tweezers while it aligns.
- b. Receiver modules
 - i. Solder the female header pins into areas J1-4 with the opening and boards facing up.
 - ii. Solder the 26 AWG stranded wire into area J5 following the color pattern Black-Red-Blue-Green-White-Yellow, bottom to top. Repeat for all receiver boards.

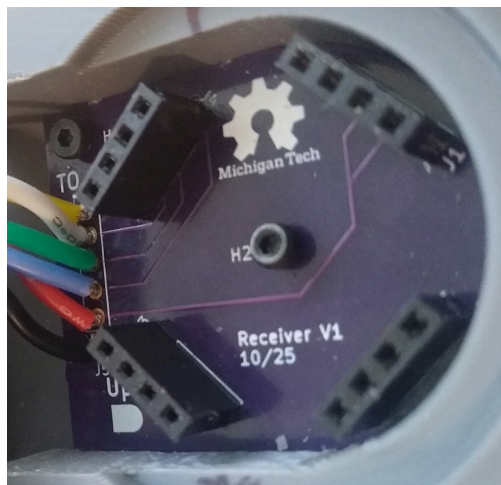


Figure 3: Receiver Module

- iii. Repeat for all receiver boards.
- c. Dev board carrier
 - i. Insert headers onto the ESP32 DevKit, place the carrier board on top, and solder.
 - ii. Solder on the buttons.
 - iii. Solder on the wires. If the wires were attached to the reserves above, the order is top (where the buttons are) -> bottom
red->blue->yellow->white->green->black

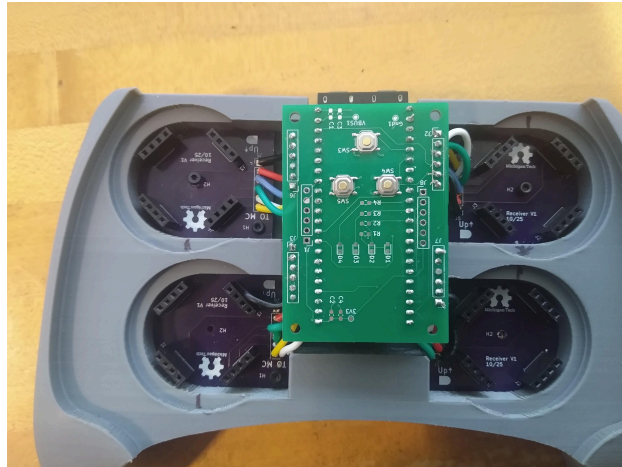


Figure 4: Dev Board Receiver Module

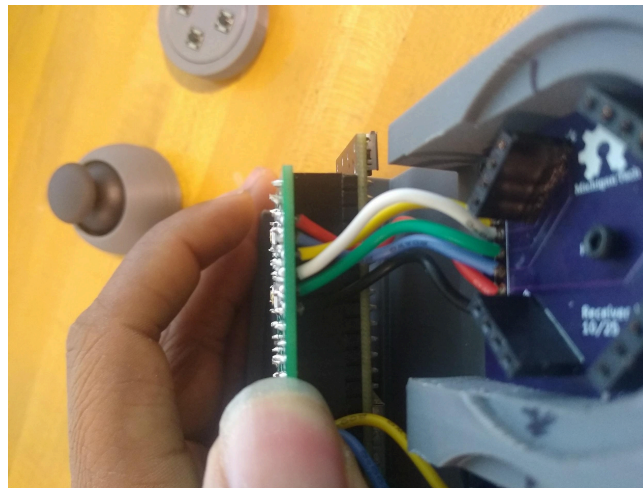


Figure 5: Connection Between Receiver Modules

- d. Custom ESP interposer
 - i. Solder the header socket to the board.
 - ii. Solder the rest of the components on the board.
The LED and accompaniment resistors are optional in this version and have no function in the code.

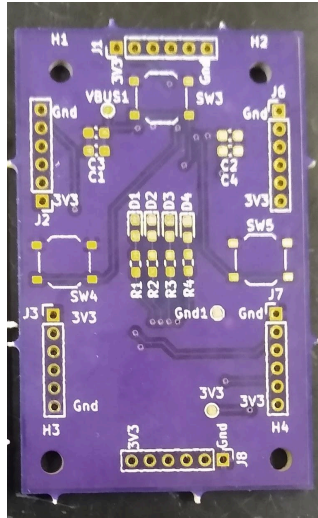


Figure 6: Receiver Module for Custom ESP32 Board

e. Joystick modules

- i. Solder the joystick into the designated positions on the PCB, keeping each button flat against the board.



Figure 7: Joystick Module (No Shell)

- ii. Insert and solder the male pin headers so that the long ends extend from the side opposite the buttons, ensuring the headers stay straight and perpendicular to the PCB.
- iii. Inspect all joints to confirm solid solder connections and verify that the buttons click freely without obstruction.
- iv. Place the module in the joystick module shell, and mark the shell with where the up arrow is pointing.
- v. Glue the top cover onto the bottom shell.



Figure 8: Joystick Module (Shell)

- f. Button modules
 - i. Solder the four push buttons into their designated labeled positions on the PCB, keeping each button flat against the board.

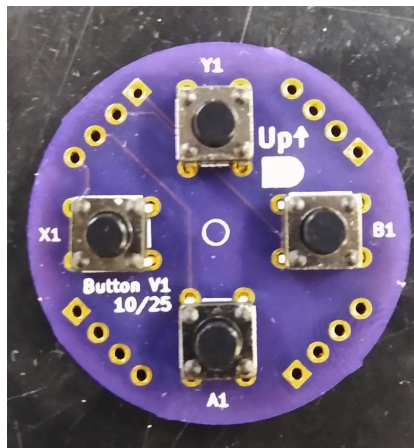


Figure 9: Button Module (No Shell)

- ii. Insert and solder the male pin headers so that the long ends extend from the side opposite the buttons, ensuring the headers stay straight and perpendicular to the PCB.
- iii. Inspect all joints to confirm solid solder connections and verify that the buttons click freely without obstruction.
- iv. Place the module in the button module shell, mark the shell with where the up arrow is pointing.

- v. Glue the top plate onto the module.

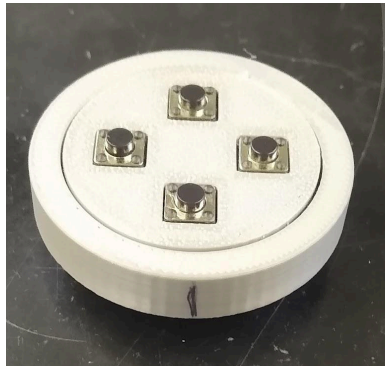


Figure 10: Button Module (Shell)

6. Uploading code

- a. Install Arduino IDE.
- b. Add ESP32 boards manager URL:
In Arduino IDE, select File -> Preferences. Paste this URL in the additional boards manager URL box:
“https://espressif.github.io/arduino-esp32/package_esp32_index.json”.
- c. Install the ESP32 board package.
Next, select “Tools” -> “Board: (whatever)” -> “Boards Manager”. Search for ESP32 and install the board package from Espressif Systems.
- d. Install the **ESP32TinyUSB** library by Dariusz.
Next, select “Tools” -> “Manage Libraries...”. Search ESP32TinyUSB and install the library by Dariusz.
- e. Import project files into Arduino IDE.
- f. Connect via USB (UART port).

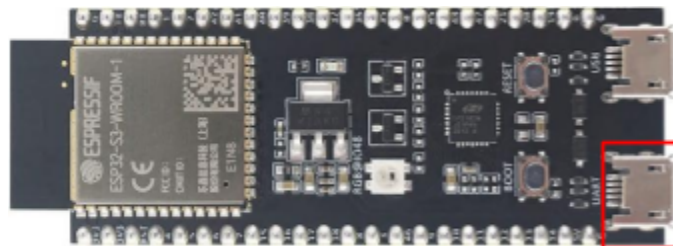


Figure 11: ESP32-S3 Dev Board

- g. Select the **ESP32-S3 Dev Module** and the correct COM port.
The COM port could be different depending on your computer.

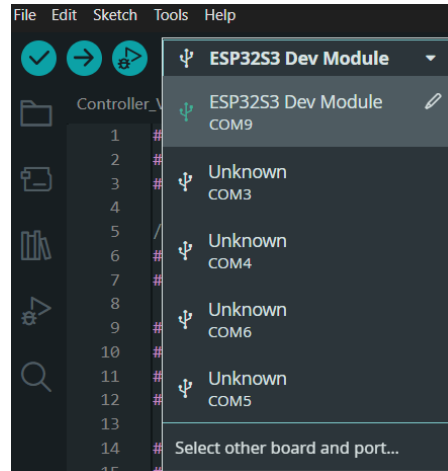


Figure 12: Arduino IDE COM Selection

- h. Select “Tools”, then change to the match following settings.

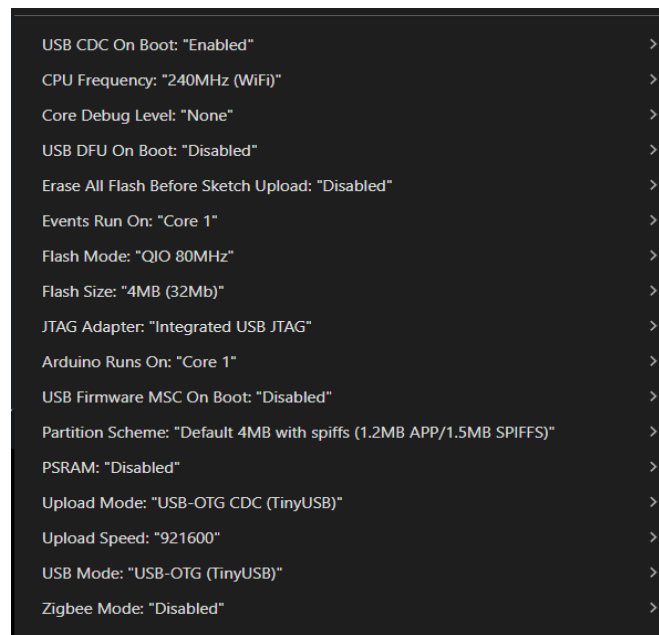


Figure 13: Board Configuration

- i. Compile and upload the sketch onto the ESP32.
 - i. Use the *UART* for programming.
 - ii. Use the *USB* port for controller operation
7. Assemble in-shell
 - a. Place receiver boards and mark screw holes on the bottom shell.
 - b. Remove the receiver boards and embed the M3 heat set inserts where you marked the bottom shell, making sure they are level with the top surface.
 - c. Replace the receiver boards and secure them with M3 screws.
 - d. Place the joystick/button modules onto the receiver boards.
 - e. Place the carrier board in the center of the bottom shell.

- f. Push the top cover into place, making sure all inputs align with their respective holes.
8. Testing
 - a. Use a program such as **Steam Input** or **gamepad-tester**.
 - b. Verify that every button and joystick direction registers properly.

Characterization Data:

Input Testing

Testing the inputs was straightforward. First was testing all the buttons. This was tested within the gamepad tester website, where the buttons were B0-B10 and the d-pad was axis9. The joysticks were tested in the same manner and were axes 0, 1, 3, and 5. From this test, the buttons were found to be shorted together and explained more in the hardware continuity section. This test also has a continuity error between the assembled controller and the code that defines which pins are to which buttons.

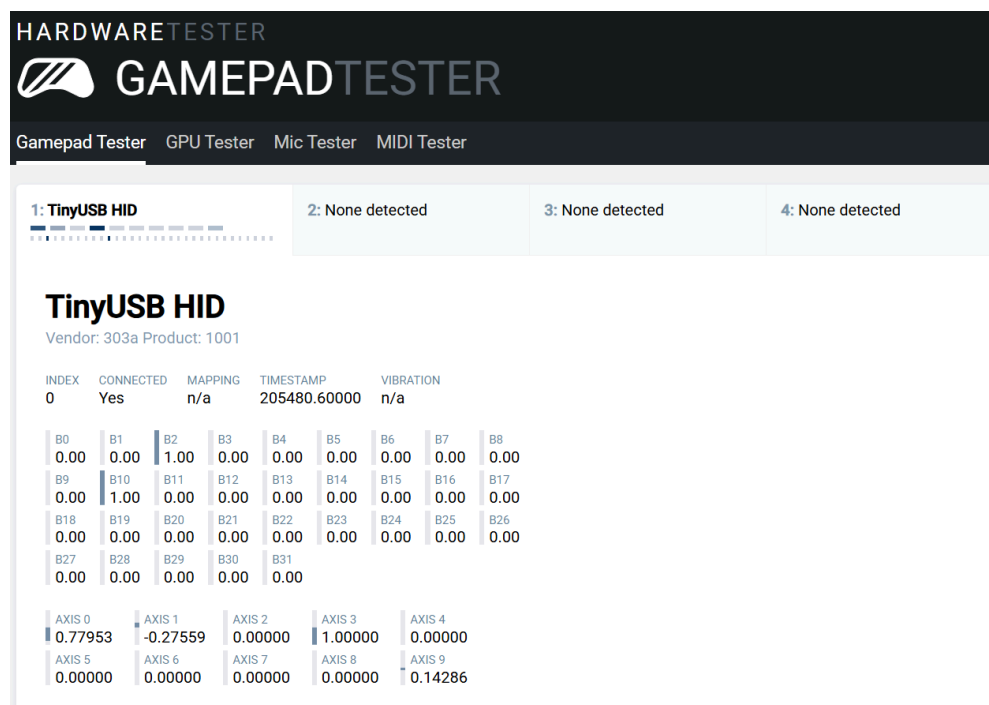


Figure 14: Gamepad Tester Example

Ergonomic Design

To see how the controller's ergonomics felt to others, we asked members of the Open Source Hardware Enterprise at Michigan Tech to hold the controller and write down their thoughts. The responses were mostly positive, with the main comment being that the size was a little large and

wide for some of the button configurations. Some of the other comments were about the finish, but this will vary depending on the 3D printer and the settings used.

Table 4: Ergonomics Feedback

Name of Reviewer	Rating from (1 low-10 high)	Comments
Evan	7	A little wide.
Ian	10	Good for big hands.
Riley	4	Too big, for small hands but hands. Holds are really nice :)
Owen	7	Really nice, maybe too big for my hands.
Garrett	7	Slightly big, otherwise very nice and comfortable
Alycia	8	Really big for my hands. But it feels nice to hold.
Joshua	8	Comfortable if not a bit big.
Emily	10	Comfortable
Katelyn	10	Comfortable, nice feel and cool design. (Maybe big but not a problem)
James	8	Comfortable. Move the lower stick buttons in.
Anonymous	6	My hands small, I like a smaller controller because I play claw

Hardware Continuity

During hardware testing, a previously undetected short was discovered on the custom ESP interposer board. The fault occurred between the 3.3 V rail and ground, creating a low-resistance path that prevented the ESP32 from booting. The short was subtle enough that most laptops' internal current protection systems did not trigger, making the issue less obvious during early debugging.



Figure 15: Custom Board Short

The problem was identified, corrected in the updated board revision, and manually repaired on the already manufactured boards. However, due to additional issues and time constraints with the custom interposer design, these original boards were ultimately scrapped for this assembly cycle. The corrected design ensures proper power isolation and restores full functionality for future revisions.

A second issue identified during continuity testing involved accidental bridging caused by stray strands in the wiring. Some of the stranded wires used during assembly had individual filaments that separated and made unintended contact with adjacent pads or traces. This was partially a workmanship issue, but was also influenced by the selected wire gauge.

To reduce the likelihood of frayed strands causing shorts, the Bill of Materials (BOM) was updated to replace the original 24 AWG wire with 26 AWG stranded wire. The smaller gauge provides better flexibility and is easier to manage during soldering, significantly lowering the risk of strand bridging in future builds.

Lastly, there is a small possibility of the joystick module shorting the 3v3 and ground rail. This was mitigated by having circular indents in the shell that prevented the module from being plugged in with this offset. But testing without the shell, this possibility was discovered.

Sponsors/Acknowledgements:

This project was currently self-sponsored by the Open Source Hardware Enterprise at Michigan Technological University.

References:

- “TinyUSB Library,” GitHub, <https://github.com/chegewara/EspTinyUSB> (accessed Dec. 5, 2025).
- “Alpakka Controller,” Input Labs, <https://inputlabs.io/alpakka> (accessed Dec. 5, 2025).
- “Gamepad Tester,” Hardware Tester, <https://hardwaretester.com/gamepad> (accessed Dec. 5, 2025).

“Proteus Controller™ kit,” ByoWave Proteus Controller,

<https://byowave.com/products/proteus-controller> (accessed Dec. 5, 2025).

“ESWAP x 2 pro controller,” ESWAP X2 PRO CONTROLLER | Thrustmaster USA,

https://eshop.thrustmaster.com/en_us/eswap-x2-pro-controller.html (accessed Dec. 5, 2025).